

좋은 질문입니다! 프로그램이 한 번 실행되고 끝나는 것이 아니라 계속 동작하게 만들려면 `while True` 무한 루프가 필수적입니다. 또한, 코드가 너무 길어지는 것을 막기 위해 기능별로 함수(`def`)를 쪼개는 것이 좋습니다. 아래처럼 크게 **3**개의 함수로 나누고, 맨 밑에 메인 루프를 두는 뼈대를 추천합니다.

Python

1. 메뉴 화면을 보여주고 입력을 받는 함수

```
def display_menu():  
    print("\n[ 체력측정 관리 시스템 ]")  
    print("1. 신규 용사 등록")  
    print("2. 전체 통계 조회")  
    print("3. 종료")  
    choice = input("메뉴를 선택하세요: ")  
    return choice
```

2. 용사 정보를 입력받고 리스트에 저장하는 함수

```
def register_soldier():  
    print("\n--- 신규 용사 등록 ---")  
    # 여기에 기존의 input()과 등급 판정, append() 로직이 들어갑니다.
```

3. 전체 통계를 보여주는 함수

```
def show_statistics():  
    print("\n--- 전체 통계 조회 ---")  
    # 여기에 len(), max(), for문 출력 등의 로직이 들어갑니다.
```

```
# =====  
# 메인 로직 (무한 루프)  
# =====  
while True:  
    user_choice = display_menu()
```

```
    if user_choice == '1':  
        register_soldier()  
    elif user_choice == '2':  
        show_statistics()  
    elif user_choice == '3':  
        print("프로그램을 종료합니다.")  
        break # 루프를 탈출하여 프로그램 종료
```

```
else
```

```
    print "잘못된 입력입니다. 다시 선택해주세요."
```

이렇게 뼈대를 잡고 나면, 메인 로직(**while**문 안쪽)이 마치 책의 목차처럼 아주 깔끔해지는 효과를 볼 수 있습니다! 기존 코드를 이 함수들 안쪽으로 옮겨 담아볼까요?

아주 핵심적인 질문입니다! 코드를 함수로 쪼갤 때 가장 중요한 것이 바로 '데이터를 넘겨주고(매개변수) 다시 받아오는(return) 과정'입니다.
말씀하신 대로 3개의 기록을 매개변수로 받고, 계산이 끝난 최종 등급을 return으로 돌려보내려면 아래와 같이 코드를 작성하시면 됩니다.

Python

1. 함수 정의: 괄호 안에 3개의 매개변수를 적어줍니다.

```
def evaluate_grade(run_time, pushups, situps):
```

```
# ... (기존의 종목별 등급 판정 if문들 들어갈 자리) ...
```

```
# 종합 등급 판정 (예시)
```

```
if pushups >= 85 and situps >= 95
```

```
    final_grade = "★최우수 특급전사★"
```

```
else
```

```
    final_grade = "특급전사"
```

```
# 2. 모든 계산이 끝난 후, 최종 결과를 밖으로 던져줍니다(return)
```

```
return final_grade
```

```
# =====
```

```
# 메인 로직 (또는 register_soldier 함수 내부)
```

```
# =====
```

```
# 사용자에게 입력받은 3개의 기록
```

```
user_run = 12.30
```

```
user_push = 80
```

```
user_sit = 90
```

```
# 3. 함수 호출: 함수 괄호 안에 입력받은 값들을 순서대로 넣어줍니다.
```

```
# 그러면 함수가 계산을 마치고 return한 값이 'overall' 이라는 변수에 쏙 들어갑니다!
```

```
overall = evaluate_grade(user_run, user_push, user_sit)
```

```
print(f"최종 부여된 등급은 {overall} 입니다.")
```

💡 포인트 요약:

1. 함수를 만들 때 괄호 안에 변수 이름(매개변수)을 뚫어놓는다.
2. 함수 맨 마지막에 return 돌려줄값을 적는다.

3. 메인 로직에서 결과를_담을_변수 = 함수이름(값1, 값2, 값3) 형태로 호출해서 사용한다.
이렇게 쓰시면 메인 코드가 몰라보게 깔끔해질 겁니다!

이 에러는 파이썬을 배우면서 전역 변수(Global Variable)와 지역 변수(Local Variable)의 개념을 짚고 넘어갈 때 반드시 만나는 아주 중요한 에러입니다!

왜 이런 에러가 날까요?

파이썬은 함수(def) 안에서 어떤 변수에 값을 더하거나 할당(+= 또는 -=)하려고 하면, 기본적으로 그 변수를 '함수 안에서만 쓰는 새로운 지역 변수'라고 착각합니다. 그래서 "어?

total_pushups라는 변수를 함수 안에서 만든 적이 없는데 왜 더하려고 하지?" 하면서

UnboundLocalError를 내뿜는 것입니다. (반면, 리스트에 .append()를 하는 것은 값을 새로 할당하는 게 아니라 안의 내용물만 바꾸는 것이라 에러가 나지 않습니다.)

어떻게 해결할까요?

함수 바깥에 있는 진짜 전역 변수를 함수 안에서 수정하고 싶다면, 파이썬에게 "내가 지금부터 쓸 변수는 여기서 새로 만든 게 아니라, 저 바깥에 있는 전역 변수야!"라고 미리 신고를 해줘야 합니다.

이때 사용하는 마법의 키워드가 바로 **global**입니다.

```
Python
# 함수 바깥에 선언된 전역 변수
total_pushups = 0

def register_soldier():
    # 파이썬에게 바깥에 있는 변수를 쓰겠다고 선언 (함수 맨 윗줄에 적어주세요!)
    global total_pushups

    pushups = int(input("팔굽혀펴기 횟수: "))

    # global 선언을 했기 때문에 이제 에러 없이 밖의 변수에 값이 누적됩니다!
    total_pushups += pushups

    print(f"현재 부대 총 팔굽혀펴기 횟수: {total_pushups}")
```

💡 포인트 요약:

함수 안에서 바깥에 있는 숫자나 문자열 변수의 값을 직접 바꾸고 싶을 때는, 함수 맨 윗줄에 **global** 변수이름을 꼭 선언해 준다!

이 부분을 적용해서 에러를 해결해 보시고, 멋지게 캡처해서 제출 자료로 활용해 보세요! 과제 마무리까지 화이팅입니다!